Lecture Notes on Advanced Algorithms and Complexity

Introduction to Algorithmic Complexity

Author: ***Abdelkrim Mebarki***
Date : ***August 17, 2026***
Master's Program in Computer Science – All Options
Department of Computer Science – Faculty of Mathematics and Computer Science
University of Science and Technology of Oran – Mohamed Boudiaf (USTO-MB)

1. Learning Objectives

These chapter provides the detailed theoretical and pedagogical background for the first lecture. The purpose is to establish the vocabulary of algorithm analysis and to develop an intuitive understanding of computational cost before introducing formal asymptotic notation such as O, Ω, and Θ.

- Define an algorithm and explain the meaning of algorithmics.
- Identify and formulate an algorithmic problem in terms of input, output, and constraints.
- Distinguish a problem from a particular problem instance.
- Describe the main stages of solving an algorithmic problem.
- Distinguish correctness, efficiency, and complexity.
- Explain intuitively the notions of input size, execution time, memory usage, and elementary operation.
- Understand why two correct algorithms for the same problem may have very different performance.
- Use simple examples to reason about scalability.
- Understand the motivation for asymptotic analysis and prepare for formal complexity analysis.

2. Fundamental Concepts

2.1 What is an Algorithm?

An algorithm is a finite, well-defined sequence of computational steps designed to solve a problem or perform a task. It specifies what must be done independently of a particular programming language or machine.

A useful way to think about an algorithm is as a recipe for computation:

- It receives some input (possibly none).
- It performs a sequence of precisely specified operations.
- It eventually terminates.
- It produces the required output.

Example — Finding the maximum value:
Given a non-empty array A, keep the first value as the current maximum. Scan the remaining values one by one. Whenever a larger value is encountered, replace the current maximum. At the end, return the stored maximum.

Available on zenodo – DOI : 10.5281/zenodo.23023380

2.2 Classical Characteristics of an Algorithm

Characteristic	Meaning	Example / Comment
Finiteness	The algorithm must terminate after a finite number of steps.	A loop must have a termination condition.
Definiteness / Precision	Every operation must be unambiguous.	"Choose a large number" is not a precise computational instruction.
Effectiveness	Each operation must be executable in finite, practical computational steps.	Compare two values; assign a value; access an array element.
Input	The algorithm may receive zero or more input values.	An array and a target value in a search problem.
Output	The algorithm produces one or more results.	Index, Sorted sequence, or Boolean answer.
Determinism	For the same input and conditions, the decisions are reproducible.	Classical deterministic algorithms behave predictably.

Modern computing also uses randomized algorithms. Therefore, determinism is best presented as a classical property of deterministic algorithms rather than an absolute requirement for every algorithm studied today.

2.3 Example: A Precise vs. Imprecise Procedure

Consider the task: "Find the largest number in a list."

Imprecise description: "Look through the list and find a very large number."

This is not sufficiently precise because "very large" does not define a decision rule.

Precise description: "Initialize max with the first element. For every remaining element x, if x > max, assign x to max. Return max."

The second description can be translated directly into a program and its behavior can be analyzed.

2.4 Algorithm, Program, and Software

Algorithm: An abstract procedure for solving a computational problem.

Program: A concrete implementation of an algorithm in a programming language.

Software: A broader system that may contain programs, libraries, interfaces, configuration, data, documentation, and services.

Example: Binary search is an algorithm.
A C++ function implementing binary search is a program component. A complete search application using binary search, a database, an interface, logging, and authentication is software.

2.5 What is Algorithmics?

Algorithmics is the systematic study of algorithms: their design, correctness, efficiency, complexity, limitations, and applications. It is concerned not only with how to obtain an answer, but also with whether the solution is correct and whether it remains practical as the problem size increases.

Typical questions in algorithmics include:

- **How** can the problem be solved?
- Is the proposed solution **correct**?
- Can the solution be **improved**?
- How much **time** does it **require**?
- How much **memory** does it **require**?
- How does its **cost** grow when the **input** becomes **larger**?
- Are there **structural properties** of the input that can be exploited?

3. The Algorithmic Problem

3.1 What is an Algorithmic Problem?

An algorithmic problem is a general computational task defined by a specification of valid inputs and the required outputs. It does not describe one particular dataset; it describes an entire class of computational situations. A problem specification normally contains three components:

- **Input specification**: What data are provided?
- **Output specification**: What result must be produced?
- **Constraints and assumptions**: What values, sizes, or structural properties are allowed?

Example — Sorting:
Input: a sequence of n comparable values.
Output: the same values arranged in nondecreasing order.
The sorting problem can be solved by insertion sort, merge sort, heap sort, quicksort, and many other algorithms.

3.2 Problem vs. Problem Instance

A problem is the general task. A problem instance is one concrete input to that task.

General problem	Concrete instance
Sort a sequence of numbers	[7, 2, 9, 4, 1]
Search for x in an array	A=[8, 3, 11, 5], x=11
Find a shortest path	A particular graph with specified vertices and edges
Multiply two matrices	Two specific matrices A and B

This distinction becomes crucial in complexity analysis because the cost of an algorithm is not fixed: it depends on the size and often the structure of the particular instance.

3.3 Examples of Algorithmic Problems

Searching: Determine whether a target value occurs in a collection, and possibly return its position.

Sorting: Rearrange values according to an ordering relation.

Maximum / Minimum: Find the largest or smallest element.

Shortest Path: Find a minimum-cost path between two vertices in a graph.

String Matching: Find occurrences of a pattern in a text.

Matrix Multiplication: Compute the product of two matrices.

Duplicate Detection: Determine whether a collection contains repeated values.

4. Solving an Algorithmic Problem

Algorithm design should not begin with writing code immediately. A disciplined workflow separates understanding the problem from implementation.

Stage	Purpose
1. Problem analysis	Understand exactly what must be computed and identify ambiguities.
2. Input identification	Determine the data supplied to the algorithm.
3. Output specification	Define precisely what constitutes a correct result.
4. Constraints and assumptions	Identify limits on input size, value ranges, ordering, memory, etc.
5. Algorithm design	Develop a general procedure that solves every valid instance.
6. Correctness reasoning	Establish that the procedure always produces the required result.
7. Implementation	Translate the abstract procedure into a programming language.
8. Testing and validation	Check behavior on representative, boundary, and adversarial cases.

9. Performance analysis	Study time and space requirements and improve the solution if needed.

4.1 Worked Example — Finding the Maximum

Problem: Given a non-empty array A of n numbers, return its maximum value.

Analysis:

- **Input**: A[0..n−1].
- **Output**: the largest value in A.
- **Constraint**: n ≥ 1.
- **Important observation**: every element may need to be considered because the maximum could be the last element.

Pseudocode:

```
MAXIMUM(A)
    max ← A[0]
    for i ← 1 to n−1 do
        if A[i] > max then
            max ← A[i]
    return max
```

Correctness intuition: after processing positions 0 through i, max stores the largest value among those processed positions. When the loop ends, all n elements have been processed, so max is the largest element in the entire array.

Example: A = [7, 2, 12, 4, 9]

- Start: max = 7
- Compare 2 with 7 → max remains 7
- Compare 12 with 7 → max becomes 12
- Compare 4 with 12 → max remains 12
- Compare 9 with 12 → max remains 12
- Return 12

4.2 Testing and Validation

Testing should not consist only of random or typical examples. Good testing deliberately targets situations that could reveal errors.

- Typical case: [4, 8, 2, 10, 5].
- Single element: [7].
- Already ordered: [1, 2, 3, 4].
- Reverse ordered: [9, 7, 5, 2].
- Repeated values: [4, 4, 4, 2].
- Negative values: [-8, -3, -10].
- Boundary case: the maximum occurs at the first element.
- Boundary case: the maximum occurs at the last element.

Testing can reveal implementation errors, but testing a finite set of examples does not prove that an algorithm is correct for every valid input.

5. Correctness, Efficiency, and Complexity

5.1 Correctness

An algorithm is **correct** if, for every valid input, it **terminates** and produces an output satisfying the problem specification.

Example:
A sorting algorithm that produces [1, 2, 3, 4] from [3, 1, 4, 2] is not necessarily correct. It must correctly sort every valid input, including duplicates, already sorted sequences, reverse order, and other cases.

5.2 Efficiency

Efficiency describes how economically an algorithm uses computational resources. In introductory analysis, the main resources are time and memory.

5.3 Complexity

Complexity provides a mathematical description of how resource requirements grow as a function of input size. It is therefore more general than measuring one execution time on one computer.

A useful mental model is: correctness asks "Does it work?", efficiency asks "How economically does it work?", and complexity asks "How does its resource usage grow as the problem gets larger?"

6. Why Analyze Algorithms?

For many computational problems, there is not a single possible algorithm. Several correct algorithms may differ dramatically in their resource requirements.

6.1 Same Problem, Different Solutions

Example: searching for a value in a collection.

Linear search checks elements sequentially. Binary search uses the ordering of a sorted array to discard approximately half of the remaining candidates after each comparison.

Both are valid solutions under their respective conditions, but their scalability is very different.

6.2 Why Input Size Changes Everything

- n = 10: even relatively inefficient algorithms may appear fast.
- n = 1 000: differences begin to become visible.
- n = 1 000 000: the choice of algorithm can determine whether a task is practical.
- n = 1 000 000 000: an algorithm with rapidly growing work may become unusable.

Therefore, algorithm analysis is fundamentally concerned with scalability.

7. Time Complexity — First Intuition

Time complexity describes how the amount of computation performed by an algorithm changes as the input size changes. It is not necessarily the exact elapsed time in seconds.

7.1 Why Stopwatch Time Is Not Enough

Suppose Algorithm A takes 0.8 seconds on one machine and Algorithm B takes 1.0 second. It would be tempting to declare A faster. However, this single measurement says little about how the algorithms behave when n becomes much larger.

Execution time depends on hardware, processor architecture, memory hierarchy, compiler optimizations, programming language, operating system, input characteristics, and implementation details.

Complexity analysis deliberately abstracts away many of these details and focuses on the growth of computational work.

7.2 Elementary Operations

A common introductory model counts elementary operations such as assignments, comparisons, arithmetic operations, and array accesses.

- x ← y : *assignment*
- x < y : *comparison*
- x + y : *arithmetic operation*
- A[i] : *array access*

The exact cost model can be refined later. For now, the important idea is that an algorithm performing 10n basic operations does substantially more work than one performing approximately 20 operations when n is very large.

7.3 Example — Summing an Array

Problem: compute the sum of n numbers.

```
SUM(A)
    s ← 0
    for i ← 0 to n−1 do
        s ← s + A[i]
    return s
```

The loop processes each of the n elements once. The amount of work therefore grows proportionally with n. We will later formalize this behavior as linear time.

7.4 Example — Nested Loops

Consider:

```
for i ← 1 to n do
    for j ← 1 to n do
        perform one elementary operation
```

The inner operation is executed n times for each of n values of i, giving $n \times n = n^2$ executions. This is the basic intuition behind quadratic growth.

For n = 100, that is 10,000 operations. For n = 10,000, it is 100,000,000 operations. The input increased by a factor of 100, but the work increased by a factor of 10,000.

8. Space Complexity — First Intuition

Space complexity describes how much memory an algorithm requires as a function of input size.

8.1 Constant Extra Space

The maximum-finding algorithm needs only a small number of additional variables, regardless of whether the array contains 10 or 10 million elements.

Its auxiliary memory therefore remains essentially constant with respect to n.

8.2 Additional Space Growing with n

Suppose an algorithm creates a second array containing the square of every input value. If the input has n elements, the additional array also contains n elements. The extra storage therefore grows with n.

When discussing space complexity, distinguish the memory occupied by the input itself from auxiliary space used by the algorithm.

9. Input Size

The choice of input-size measure is fundamental. There is not always one universal definition of n.

Problem type	Natural size measure
Array	Number of elements, n
String	Number of characters, n

Graph	Often |V| vertices and |E| edges
Matrix	Dimensions such as m × n
Integer arithmetic	Sometimes the number of bits needed to represent the integer

For example, a graph with 100 vertices can have very different computational characteristics depending on whether it has 100 edges or close to 4,950 edges. Thus |V| alone may not describe the input sufficiently.

10. A Detailed Example — Searching an Array

10.1 Problem Statement

Given an array A and a target value x, determine whether x occurs in A. If it occurs, return an index; otherwise return −1.

10.2 Linear Search

```
LINEAR-SEARCH(A, x)
    for i ← 0 to n−1 do
        if A[i] = x then
            return i
    return −1
```

Linear search makes no assumption that the array is sorted. It examines elements sequentially until it finds the target or reaches the end.

10.3 Best and Worst Cases

- **Best case**: x is at the first position. In this case, one comparison may be sufficient.
- **Worst case**: x is absent or at the last position. In this case, n comparisons may be required.
- Intermediate cases require a number of comparisons depending on the target position.

10.4 Binary Search

If A is sorted, we can exploit ordering. Compare x with the middle element. If x is smaller, only the left half can contain x; if x is larger, only the right half can contain x. Repeat on the remaining half.

```
BINARY-SEARCH(A, x)
    n ← length(A)
    low ← 0
    high ← n−1
    while low ≤ high do
        mid ← low + floor((high - low)/2)
        if A[mid] = x then return mid
        else if A[mid] < x then low ← mid + 1
             else high ← mid − 1
    return −1
```

10.5 Why Binary Search Is Powerful

For n = 1 000 000, linear search may need to inspect up to 1,000,000 elements. Binary search needs only about 20 halving steps because 2^20 = 1,048,576.

This is a central lesson of algorithm analysis: exploiting structure in the input can transform the scalability of a solution.

Important limitation: binary search requires an ordered structure and appropriate random access. It is not simply a faster version of linear search that can always be substituted.

11. Best Case, Worst Case, and Average Case

Algorithm performance may depend on the particular input instance, not only on its size.

Best case: The most favorable valid input or execution scenario.

Worst case: The most costly valid input or execution scenario.

Average / expected case: Performance averaged according to an explicitly assumed distribution or randomness model.

For linear search, the best case is finding x immediately. The worst case occurs when x is absent or at the final position. Worst-case analysis is particularly useful because it provides a guaranteed upper bound on the work required.

12. Scalability: Why Growth Matters

Consider an illustrative machine capable of performing approximately one billion elementary operations per second. The following figures are only a conceptual model, not a promise of real execution time:

Complexity Pattern	n=1,000 (103)	n=1,000,000 (106)	n=1,000,000,000 (109)
Logarithmic $\log_2 n$	≈10 operations 10 ns	≈20 operations 20 ns	30 operations 30 ns
Linear n	1,000 operations 1 µs	$1,000,000 operations 1 ms	1,000,000,000 operations 1 s
Linearithmic $n \log_2 n$	≈10,000 operations 10 µs	≈20,000,000 operations 20 ms	≈30,000,000,000 operations 30 s
Quadratic n^2	1,000,000 operations 1 ms	1,000,000,000,000 operations 16.7 minutes	1,000,000,000,000,000,000 operations 31.7 years

The point is not the exact numbers. The point is the dramatic difference in growth. An algorithm that looks perfectly acceptable for n = 1,000 may become impractical for n = 1,000,000.

13. Introduction to Asymptotic Analysis

Asymptotic analysis studies how the cost of an algorithm grows as the input size becomes large. It focuses on the dominant growth behavior rather than exact machine-dependent constants.

13.1 Typical Growth Patterns

Growth type	Example	Intuition
Constant	1	Cost is essentially independent of n.
Logarithmic	log n	The problem size is repeatedly reduced by a factor, often 2.
Linear	n	Work grows proportionally with input size.
Quadratic	n^2	Often produced by two nested loops over the input.
Cubic	n^3	Often produced by three nested loops.
Exponential	2^n	Grows extremely rapidly; often appears in brute-force search.

At this stage, students should recognize these growth patterns conceptually. Formal definitions of O, Ω, and Θ will be introduced later.

14. A Second Worked Example — Comparing Two Approaches

Problem: determine whether an array contains any duplicate value.

Approach A — Compare Every Pair

Compare each element with every later element. If any pair is equal, a duplicate exists.

For n elements, there are roughly n(n−1)/2 pairs. The number of comparisons therefore grows quadratically.

```
Algorithm: HasDuplicatesBruteForce(A)
Input: An array A of n elements
Output: True if A contains at least one duplicate, False otherwise

1. For i = 0 to n - 2 do:
2.     For j = i + 1 to n - 1 do:
3.         If A[i] == A[j] then:
4.             Return True
5. Return False
```

Approach B — Use a Set

Scan the array once. For each value, check whether it has already been seen; if not, insert it into a set.

Under the usual expected-time assumptions for hash tables, this can reduce the running time dramatically, at the cost of additional memory.

```
Algorithm: HasDuplicatesHashSet(A)
Input: An array A of n elements
Output: True if A contains at least one duplicate, False otherwise
1. Initialize an empty Hash Set 'seen'
2. For each value in A do:
3.     If value is in seen then:
4.         Return True
5.     Insert value into seen
6. Return False
```

This example illustrates an important trade-off: an algorithm may use more space in order to reduce time.

15. Important Conceptual Distinctions

Distinction	Key idea
Correctness vs. **efficiency**	A slow algorithm can still be correct.
Algorithm vs. **implementation**	The same algorithm may have multiple implementations.
Execution time vs. **complexity**	Execution time is a measurement; complexity describes growth.
Input size vs. **input value**	Large numerical value doesn't necessarily mean large input representation.
Problem vs. **instance**	The problem is general; the instance is one concrete input.
Testing vs. **proof**	Testing finds errors but does not establish universal correctness.

16. Common Misconceptions

Misconception	Correction
"The **fastest** program on **my computer** is the best algorithm."	A single benchmark does not describe scalability or behavior on different hardware and input sizes.
"If an algorithm is **correct**, it is **good**."	Correctness is necessary but not sufficient; efficiency and scalability also matter.

"O(n) means the algorithm **always takes n seconds**."	Asymptotic notation describes growth, not seconds.
"Binary search is **always better** than linear search."	Binary search requires suitable ordering and access conditions; linear search may be preferable in other situations.
"**Testing** proves **correctness**."	Testing increases confidence but cannot cover infinitely many possible inputs.
"**More memory** is always **bad**."	Extra memory can be intentionally exchanged for faster computation.

17. Classroom Exercises

1. Write an algorithm in plain language for finding the minimum value in a non-empty array.
2. Give three different valid instances of the sorting problem.
3. Explain the difference between an algorithm and a program using an example from a programming language you know.
4. For a linear search, identify a best-case and a worst-case input.
5. Consider a loop that performs one operation for each element of an array. How does the number of operations change when n doubles?
6. Consider two nested loops, each running from 1 to n. Explain why the inner operation is executed n^2 times.
7. Give an example where additional memory could be used to reduce execution time.
8. Explain why the input-size parameter for a graph may need both |V| and |E|.

18. Summary

Algorithmic complexity is introduced to answer a fundamental question: how does the computational cost of an algorithm change when the problem becomes larger? Before formal mathematical analysis, students need a clear conceptual vocabulary.

- An algorithm is a finite and precise computational procedure.
- An algorithmic problem specifies valid inputs and required outputs.
- An instance is one concrete input to a general problem.
- Algorithm design should precede implementation.
- Correctness, efficiency, and complexity are related but distinct concepts.
- Input size is a key parameter in algorithm analysis.
- Time complexity concerns computational work; space complexity concerns memory requirements.
- Different algorithms for the same problem can have radically different scalability.
- Linear search and binary search provide a simple illustration of this phenomenon.
- Asymptotic analysis focuses on growth as input size becomes large.
- Formal O, Ω, and Θ notation will provide the mathematical language needed for the next stage of the course.

19. Transition to the Next Course

The next stage is to replace qualitative statements such as "this algorithm grows linearly" or "this one grows quadratically" with precise mathematical tools. Students will learn how to count operations, construct cost functions, simplify them, and express upper, lower, and tight asymptotic bounds using O, Ω, and Θ.

20. Further Reading and Recommended Resources

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, ***Introduction to Algorithms***, 4th ed. Cambridge, MA, USA: MIT Press, 2022, ISBN: 978-0-262-04630-5.

[2] A. Levitin, ***Introduction to the Design and Analysis of Algorithms***, 3rd ed. Boston, MA, USA: Pearson, 2012, ISBN: 978-0-13-231681-1.

[3] S. S. Skiena, ***The Algorithm Design Manual***, 3rd ed. Cham, Switzerland: Springer, 2020, ISBN: 978-3-030-54255-9.

[4] R. Sedgewick and K. Wayne, ***Algorithms***, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2011, ISBN: 978-0-321-57351-3.

[5] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, ***Data Structures and Algorithms in Java***, 6th ed. Hoboken, NJ, USA: Wiley, 2014, ISBN: 978-1-118-77331-1.

[6] J. Kleinberg and É. Tardos, ***Algorithm Design***. Boston, MA, USA: Addison-Wesley, 2005, ISBN: 978-0-321-29535-4.

[7] G. Brassard and P. Bratley, ***Fundamentals of Algorithmics***. Upper Saddle River, NJ, USA: Prentice Hall, 1996, ISBN: 978-0-13-335068-5.

[8] D. E. Knuth, ***The Art of Computer Programming****, Volume 1: Fundamental Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997, ISBN: 978-0-201-89683-1.

[9] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, ***The Design and Analysis of Computer Algorithms***. Reading, MA, USA: Addison-Wesley, 1974, ISBN: 0-201-00029-6.

[10] N. Wirth**, *Algorithms + Data Structures = Programs***. Englewood Cliffs, NJ, USA: Prentice-Hall, 1976, ISBN: 978-0-13-022418-7.